Software Design
Unified Modeling Language

FH JOANNEUM
Software Design & Cloud Computing

Teiniker/Schindler/Schalk
Version: 26

CC BY SA

Unified Modeling Language

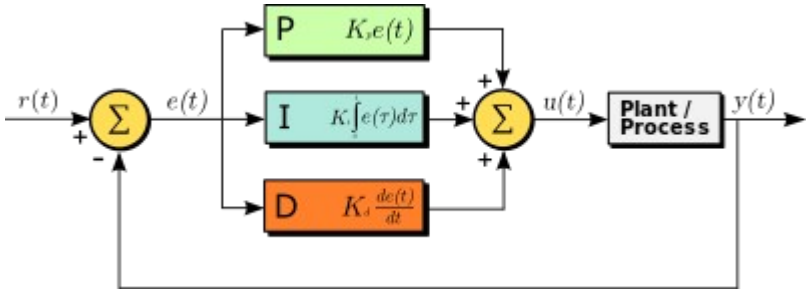
Outline

- Introduction
- Objects and Classes
- Class-Diagram
- Package-Diagram
- Object-Diagram
- Sequence-Diagram

Introduction

Why Modeling?

Engineers build models to find out if their designs will work.

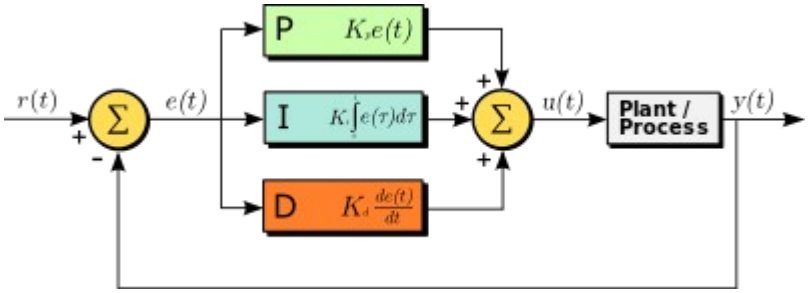


This implies that **models must be testable**. If they can't evaluate the model, the model has no value.

Engineers investigate designs with models when the models are **much cheaper than the real thing** they are building.

Introduction

Why Modeling?



- There is **no firm criteria for testing** a UML diagram.
- We can look at it (**applying principles and patterns to it**) but this evaluation is still very subjective.
- There are times when it's **easier to change source code** than to change a diagram.

Introduction

UML Overview

The **Unified Modeling Language (UML)** is a family of graphical notations, backed by a single meta-model, that help in describing and designing software systems.

- UML is an open standard, controlled by the **Object Management Group (OMG)**.
- Note that programming languages are not at a high enough **level of abstraction** to facilitate discussions about design.

UML Overview

UML 2 describes 13 diagram types which can be classified in the following way:

- **Structure Diagrams:**
  - **Class Diagram**
  - **Object Diagram**
  - Component Diagram
  - Composite Structure Diagram
  - Deployment Diagram
  - **Package Diagram**

Introduction

UML Overview

- **Behavior Diagrams:**
  - Activity Diagram
  - Use Case Diagram
  - State Diagram
  - Interaction Diagrams
    - **Sequence Diagram**
    - Communication Diagram
    - Interaction Overview Diagram
    - Timing Diagram

Effective Use of UML

We can not simply use UML in the way classic engineers use blueprints and models.

When should we use UML:

- **Communicating with others**
  UML is convenient for communicating design concepts among software developers. It depicts a nice overview for newcomers.
- **Road maps**
  UML can be useful for creating road maps of large software structures. Such diagrams capture the knowledge that all the developers must keep in their heads in order to work effectively in the system.

Effective Use of UML

When should we use UML: (continued)

- **What to keep and what to throw away**
  Most of the UML diagrams should be short-lived. Write them on a white board, or on scraps of paper.
  The real useful diagrams will keep showing up over and over again. Keep only those whose long-term survival has lots of value.

FAQs

… from the Introduction

- What **UML diagram types** did we use in the class and in which two groups can they be divided?
- Describe an **example of using** a class-, package-, object-, and sequence diagram in software development practice.
- Describe the relation between **UML diagram types** and a **software model**.
- Describe examples of an **effective use of UML diagrams**.

Objects and Classes

Outline

- Objects
- Classes
- Object Oriented Modeling

Objects and Classes

What are Objects?

An **object** is a discrete entity with a well-defined boundary that encapsulates state and behavior.

The following properties are common to all objects:

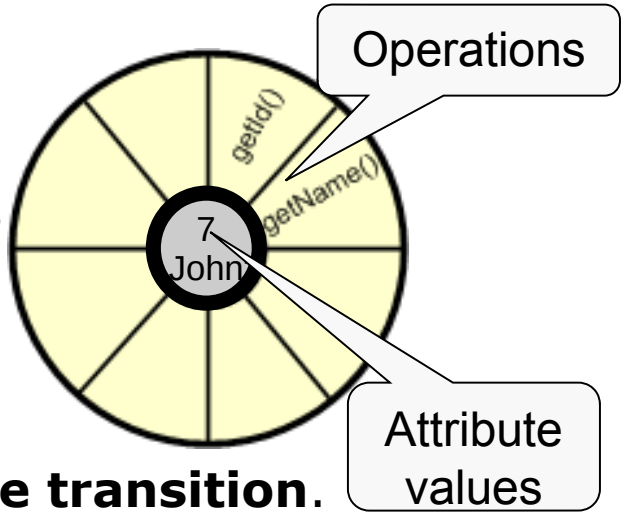
- **Identity**: this is the object's unique existence in time and space. It is what makes it different from all other objects.
- **State**: this is determined by the attribute values of an object and the relationships the object has to other objects at a particular point of time.
- **Behavior**: an operation is the specification of a piece of behavior. An implementation of that behavior is called a method.

Objects and Classes

Encapsulation

The state of this object is the set of attribute values held by the object at any point in time.

- We can **change an attribute's value** only by executing an object's operation.
- **Data-hiding** is one of the primary benefits of OO programming, and it can lead to more robust and expandable software.
- Any operation invocation that leads to a change in the object's state is called a **state transition**.

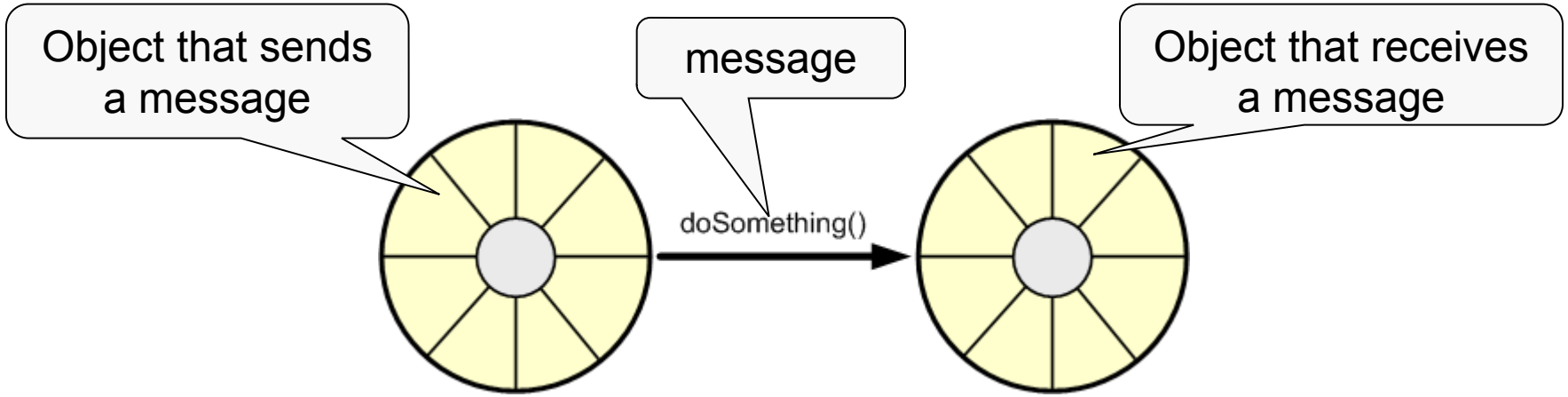


Objects and Classes

Messaging

Objects collaborate to perform the functions of the system.

What this means is that **objects form links to other objects** and **send messages** back and forth along those links.



Objects and Classes

What are Classes?

A **class** is a descriptor for a set of objects that share the same attributes, operations, methods, relationships and behavior.

We can think of a class as being a **template for objects**.
A class determines the structure (set of features) of all objects of that class:

- Every object is an **instance of exactly one class**.
- All objects of the **same** class must have the **same** set of operations, the **same** set of attributes, and the **same** set of relationships, but may have **different** attribute values.

Objects and Classes

Object-Oriented Modeling

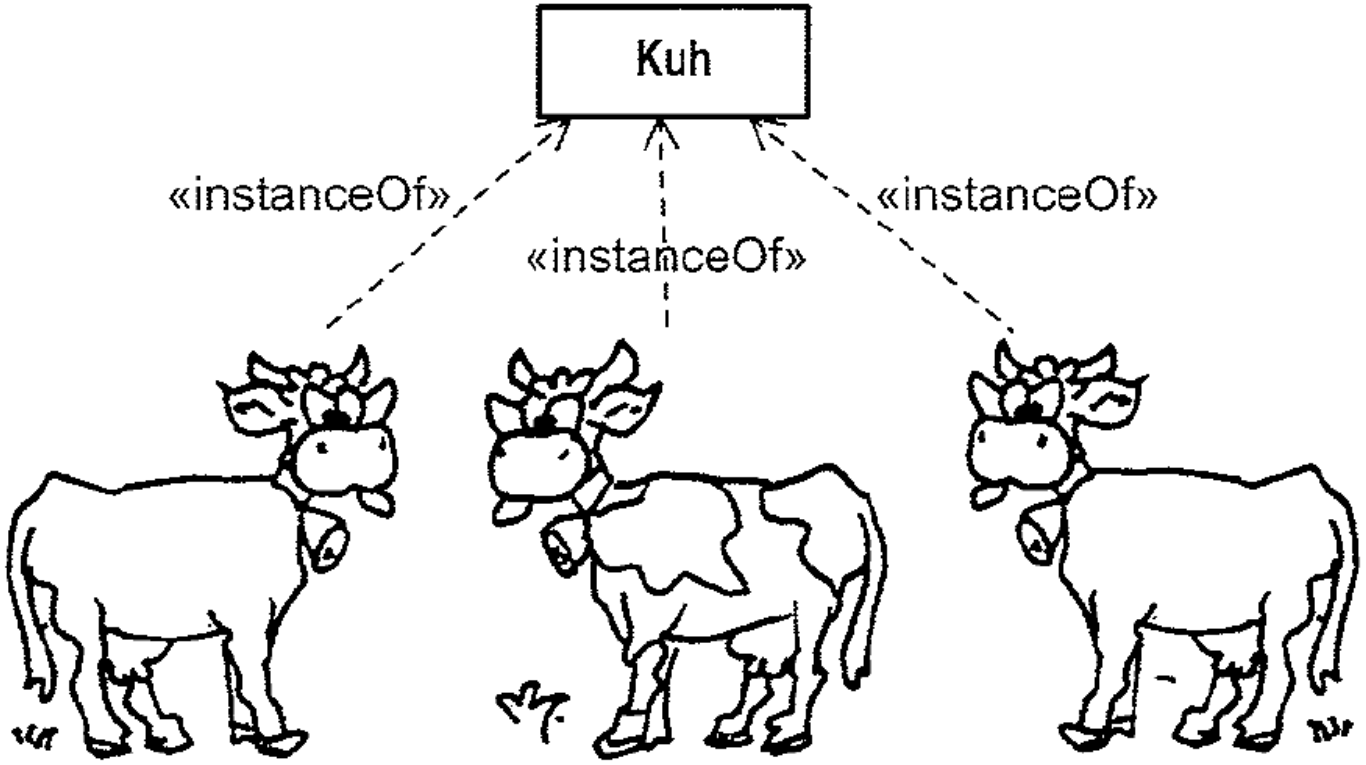
Classification is possibly the single most important way that human beings have of **ordering information about the world**.

Classes allow us to describe the **set of features** that every object of the class must have without having to describe every one of those objects.

Given that there are so many options, **choosing the most appropriate classification scheme** is one of the most important aspects of object oriented analysis and design.

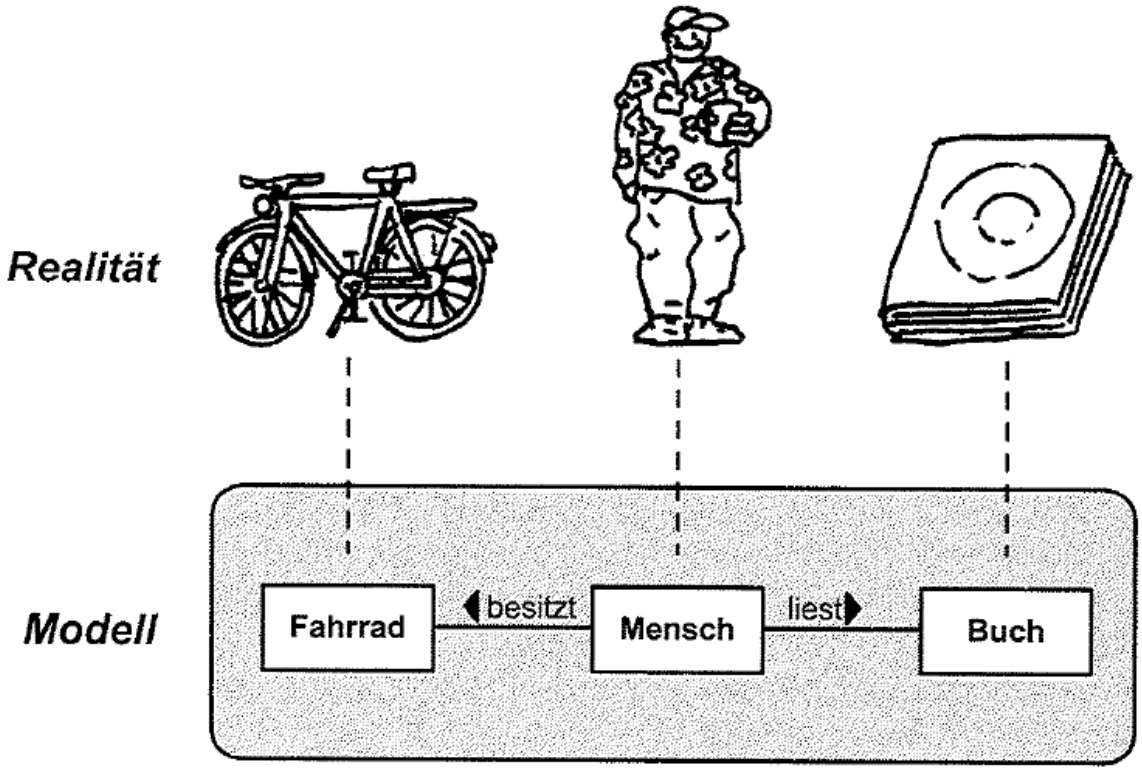
Objects and Classes

Object-Oriented Modeling



Objects and Classes

Object-Oriented Modeling



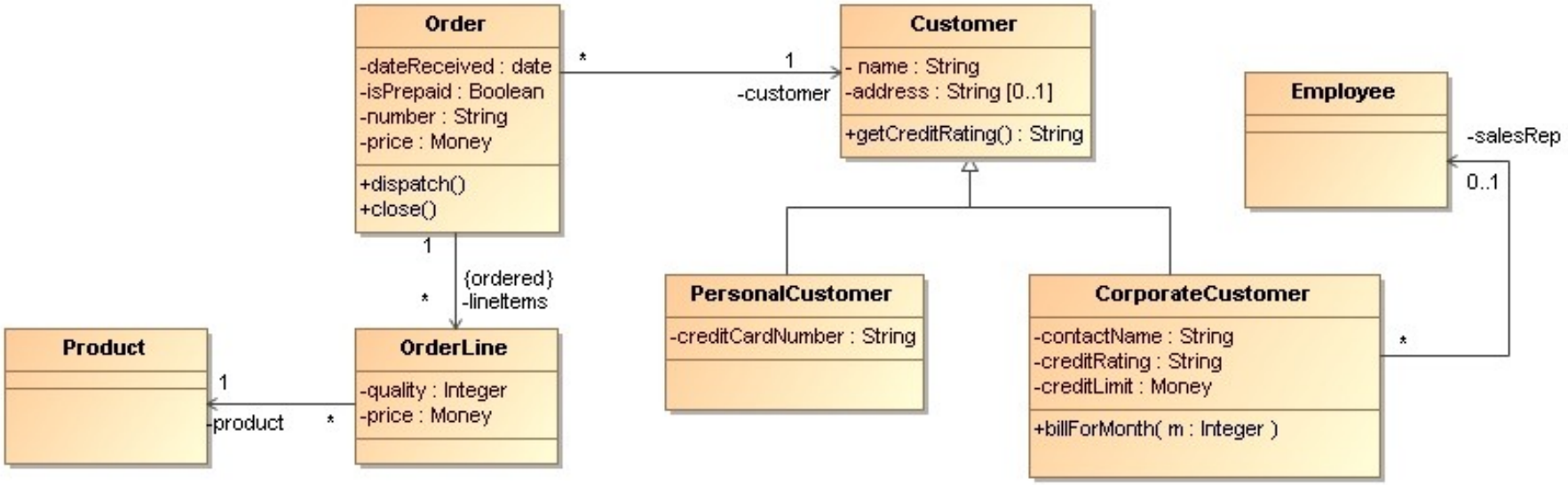
Class Diagram

Outline

- Overview
- Class
- Stereotypes
- Properties
- Multiplicity
- Visibility
- Operations
- …

Class Diagram - Overview

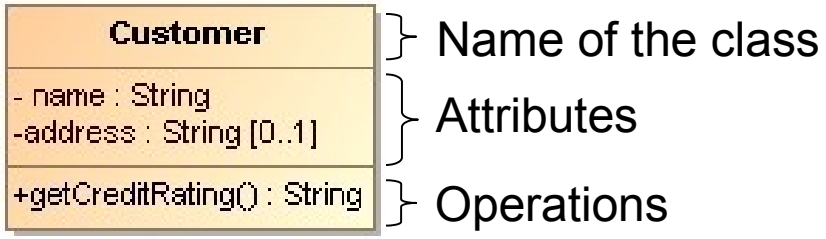
A **class diagram** describes the types of objects in the system and the various kinds of static relationships that exist among them.



Class Diagram

Class

The UML notation for classes specifies the name of a class, its attributes and operations.



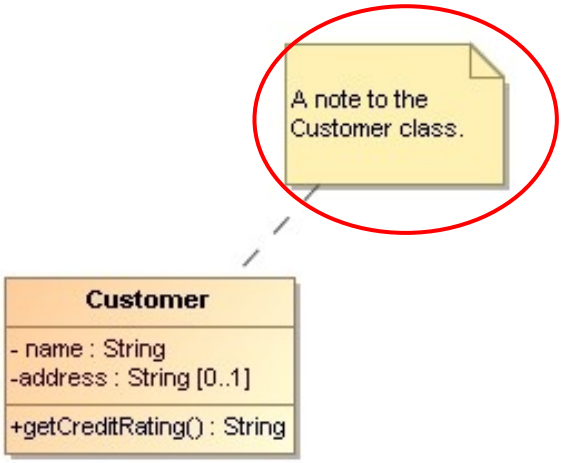
The only mandatory part of this definition is the name of the class. All other parts are optional.

Classes represents “things”, so they should have a name that is a noun or a noun phrase.

Class Diagram

Notes

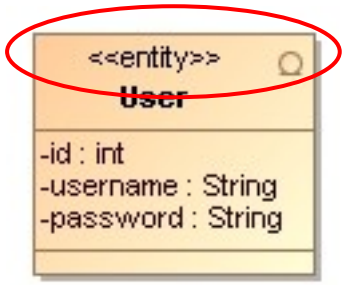
Notes are comments in the diagrams.
Notes can stand on their own, or they can be linked with a dashed line to the elements they are commenting.



Class Diagram

Stereotypes

Stereotypes allow us to **define a new UML modeling element** based on an existing one. We define the semantics of the stereotype ourselves.



Each model element can have **zero to many stereotypes**. Notice that we can stereotype relationships as well as classes.

Class Diagram

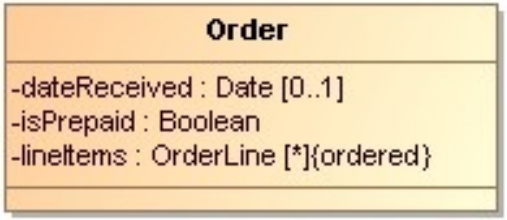
Properties

Properties represent structural features of a class.

We can think of properties as corresponding to fields in a class.

Properties appear in two quite distinct notations:

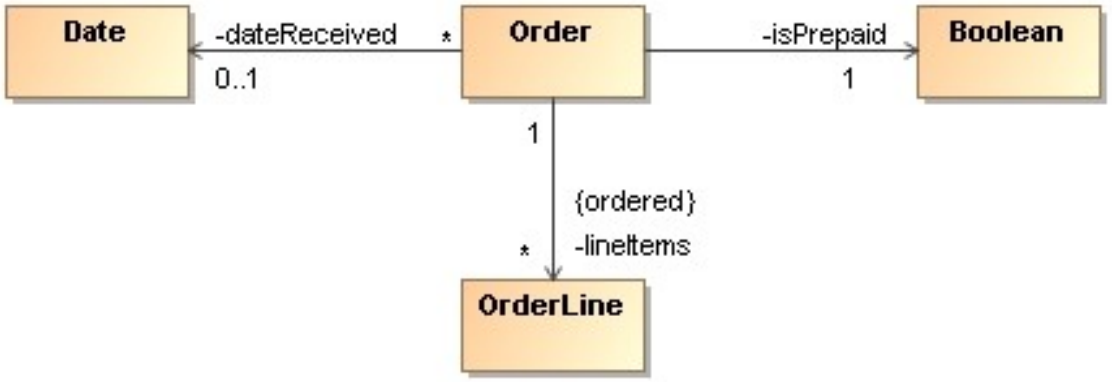
1) **Attributes.** The attribute notation describes a property as a line of text as well as as type within the class box itself.



Class Diagram

Properties

2) **Associations.** An association is a solid line between two classes, directed from the source class to the target class.



We use attributes for small things (e.g. Booleans), and associations for more significant classes (e.g. Order).

Class Diagram

Multiplicity

The multiplicity of a property is an indication of how many objects may fill the property.

The most common multiplicities are:

- **[1]** exactly one
- **[0..1]** zero or one (optional)
- **[*]** zero or more

By default, the elements in a multi-valued multiplicity form a set. If the ordering has meaning, we need to add **{ordered}** to the association end.

Class Diagram

Visibility

A class has public or private elements.

Public elements can be used by any other class

Private elements can be used only by the owning class

Package-private **elements** can only be seen and used by the package in which it was declared. This is the default in Java.

Protected elements equals the package-private visibility, plus it can be seen by subclasses.

Class Diagram

Visibility

Attributes or operations can be tagged with a visibility indicator.

The most common visibility tags are:

- **`public`** **`+`**
- **`private`** **`-`**
- **`Package`** **`~`**
- **`protected`** **`#`**

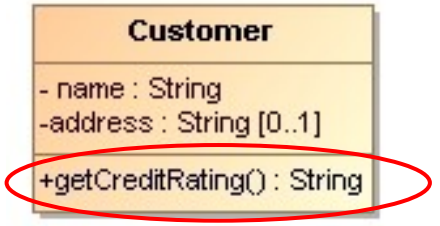
Class Diagram

Operations

Operations are the actions that a class knows to carry out. Operations most obviously correspond to the methods on a class.

The full UML syntax for operations is:

- visibility marker: public (+), private (-)
- name
- parameter-list
- return-type
- property-string e.g. {ordered}



Class Diagram

Static Operations and Attributes

UML refers to an operation or an attribute that belongs to a class rather than to an instance as **static**.

This is equivalent to static members in object oriented programming languages.

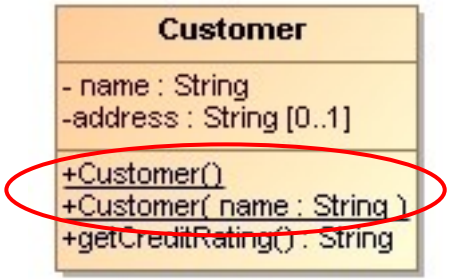


Static features are underlined on a class diagram.

Class Diagram

Constructors

Constructors are special operations that create new instances of classes – these operations must be **class scoped**.



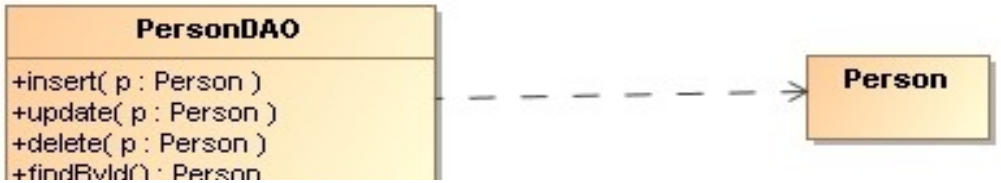
There are different standards for naming constructors:

- A completely generic approach is just to call the constructor **create(...)**.
- Java, C#, and C++ all demand that the constructor name be the same as the class name.

Class Diagram

Dependencies

A dependency exists between two elements if changes to the definition of one element may cause changes to the other.



The UML has many varieties of dependency, each with particular semantics and keywords (**<<create>>**, **<<use>>**, etc.)

Class Diagram

Bidirectional Associations

A bidirectional association is a pair of properties that are linked together as inverses.

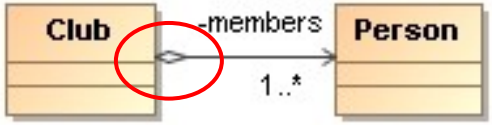


Implementing a bidirectional association in a programming language is often a little tricky because we have to be sure that both properties are kept synchronized.

Class Diagram

Aggregation

An aggregation is a **part-of relationship**.



Aggregation is an association where the entities are configured together to create a more complex object, describing the **relationship between the complex objects and its entities**.

In contrast to a composition, a part can also exist **without** the complex object.

Entities, as a part, can also be part of multiple complex objects.

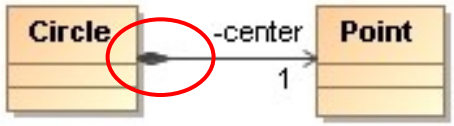
UML includes aggregation but with hardly any semantics. Therefore, we can say that aggregation is a “modeling placebo”.

Advice: If in doubt, use a plain association

Class Diagram

Composition

A composition, as a **kind of association**, is a good way of showing properties that own by value, or properties that have a **strong and exclusive ownership** of particular other components.

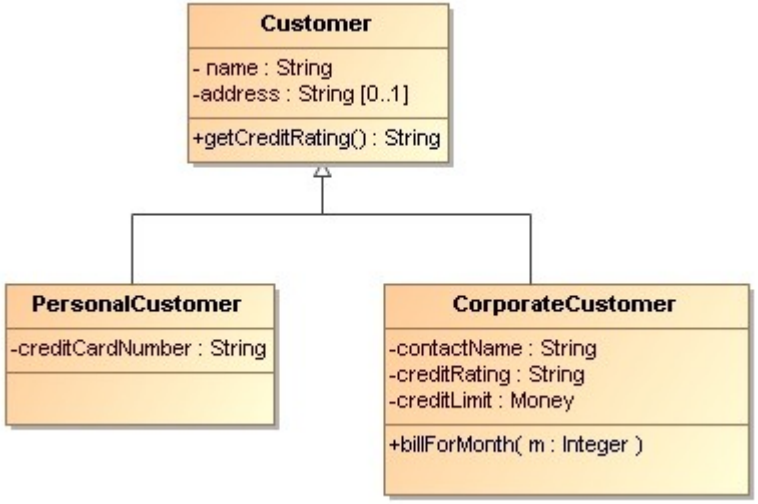


The "**no sharing**" rule is the key to composition!
If we delete circle, it should automatically ensure that the owned point is also deleted.

Class Diagram

Generalization

The subclass inherits all the features of the superclass and may override any superclass methods.



Class Diagram

Class vs. Interface Inheritance

Class inheritance defines an object's implementation in terms of another object's implementation. It inherits behavior from a base.

Interface inheritance (or subtyping) describes when an object can be used in place of another. It inherits the description of behavior.

Class Diagram

Inheritance vs. Parameterized Types

Another technique for reusing functionality is through parameterized types, also known as **generics** (Java) and **templates** (C++).

This technique lets us define a type without specifying all the other types it uses. The **unspecified types are supplied as parameters** at the point of use.

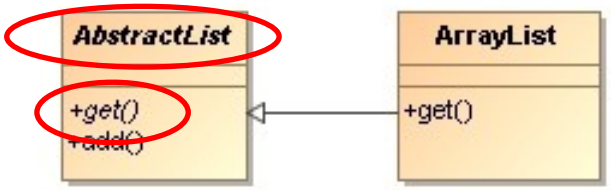
```
public interface List<E> extends Collection<E> {
boolean add(E e);
};
```

Neither inheritance nor parameterized types can change at run-time!

Class Diagram

Abstract Class

An abstract class is a class that cannot be directly instantiated. Instead, we instantiate an instance of a subclass.

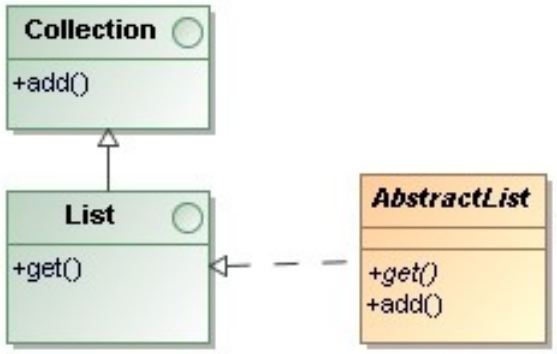


Typically, an abstract class has **one or more operations that are abstract**. An abstract operation has no implementation, it is pure declaration.

Class Diagram

Interface

An interface has no implementation, all its operations are abstract.



An interface is marked with a special sign.

Class Diagram

Abstract Class vs. Interface

An **abstract class** has one or more operations that are abstract. An **abstract operation** has no implementation. An abstract class cannot be directly instantiated, we have to instantiate an instance of a subclass.

An **interface** has no implementation, all its features are abstract. A class can provide an interface by implementing it.

To be continued ...

In part 2

References

- *Martin Fowler*
  UML Distilled
  Addison-Wesley, 3rd Edition, 2004
- *Bernd Östereich*
  Objektorientierte Sofwareentwicklung
  Analyse und Design mit UML 2.1
  Oldenbourg, 8. Auflage, 2006
- *Jim Arlow, Ila Neustadt*
  UML 2 and the Unified Process
  Addison-Wesley, 2nd Edition, 2005
- *Robert C. Martin*
  UML for Java Programmers
  Prentice Hall, 2003